



Robótica: Relatório de nivelamento

Discente: Adhemar Luiz Cavalcanti Silva Rocha Neto

Depois do estudo de toda a base teórica do método da Janela Dinâmica (DWA) no relatório anterior (baseado no artigo de Fox), o novo objetivo era: **fazer o robô movimentar-se na prática dentro do simulador**.

Para isso, modifiquei toda a estrutura criada antes e reuni tudo em um script em Python (main_robo.py) para controlar o robô à distância e simplificar e centralizar toda a lógica. A meta era simples: o robô tinha de ler a sua própria posição no mapa, descobrir a coordenada do alvo (Target) e navegar sozinho até lá. Como era um teste inicial para validar o movimento, decidi simplificar o algoritmo: deixei o sensor de desvio de obstáculos desligado e foquei o código apenas em calcular a melhor velocidade e a melhor curva para alinhar o robô diretamente com o alvo.

2. O código desenvolvido

Este foi o código mais limpo e estruturado que desenvolvi para o teste. Ele tenta simular o arco de trajetória e escolher o melhor comando a cada ciclo de tempo ($DT = 0.1s$):

```
import math

import time

from coppeliasim_zmqremoteapi_client import RemoteAPIClient


# Dimensões físicas do robô iCreate 2

R = 0.036      # Raio da roda [m]

L = 0.235      # Distância entre as rodas [m]

DISTANCIA_ALVO = 0.15 # Tolerância de chegada perto do alvo [m]


# Limites da minha Janela Dinâmica

MAX_V = 0.15    # Velocidade máxima para a frente [m/s]

MAX_W = 1.2     # Velocidade máxima de giro [rad/s]

DT = 0.1        # Tempo de previsão do próximo passo [s]


def normalizar_angulo(angulo):

    """Garante que o ângulo fique sempre entre -pi e +pi"""

    return math.atan2(math.sin(angulo), math.cos(angulo))
```

```

def main():

    print("A ligar ao CoppeliaSim...")

    client = RemoteAPIClient()

    sim = client.require('sim')

    print("Conectado com sucesso!")


    # Mapeamento dos objetos da cena

    try:

        robot = sim.getObject('/Cuboid')

        motor_esq = sim.getObject('/Cuboid/MOTOR_ESQUERDO')

        motor_dir = sim.getObject('/Cuboid/MOTOR_DIREITO')

        alvo = sim.getObject('/Target')

    except Exception as e:

        print(f"Erro ao encontrar o objeto na cena: {e}")

        return


    client.setStepping(True)

    sim.startSimulation()


    print("\n--- SESSÃO DE TESTES INICIADA ---")


    try:

        while True:

            # 1. Ler os dados de posição global (-1) no simulador

            pos_robo = sim.getObjectPosition(robot, -1)

            pos_alvo = sim.getObjectPosition(alvo, -1)

```

```
ori_robo = sim.getObjectOrientation(robot, -1)
```

```
# Ajuste de direção do robô (correção geométrica de 180 graus)
```

```
theta = normalizar_angulo(ori_robo[2] + math.pi)
```

```
# 2. Calcular a distância e o erro de rumo até ao Target
```

```
dx = pos_alvo[0] - pos_robo[0]
```

```
dy = pos_alvo[1] - pos_robo[1]
```

```
distancia = math.hypot(dx, dy)
```

```
angulo_alvo = math.atan2(dy, dx)
```

```
erro_angular = normalizar_angulo(angulo_alvo - theta)
```

```
# Condição de parada (Sucesso)
```

```
if distancia < DISTANCIA_ALVO:
```

```
    sim.setJointTargetVelocity(motor_esq, 0)
```

```
    sim.setJointTargetVelocity(motor_dir, 0)
```

```
    print("\n🎯 [SUCESSO] O robô chegou ao alvo!")
```

```
    break
```

```
# 3. Lógica de busca do DWA (Testar velocidade para escolher a melhor)
```

```
melhor_v = 0.0
```

```
melhor_w = 0.0
```

```
menor_erro_futuro = float('inf')
```

```
v_teste = 0.0
```

```

while v_teste <= MAX_V:

    w_teste = -MAX_W

    while w_teste <= MAX_W:

        # Ignora se a opção simular o robô totalmente parado

        if v_teste == 0.0 and abs(w_teste) < 0.15:

            w_teste += 0.15

            continue

        # Predição Cinematica: Onde o robô estará no próximo ciclo?

        theta_futuro = normalizar_angulo(theta + w_teste * DT)

        x_futuro = pos_robo[0] + v_teste * math.cos(theta_futuro) * DT
        y_futuro = pos_robo[1] + v_teste * math.sin(theta_futuro) * DT

        # Função de Custo: Mede o erro de rumo nessa posição simulada

        dx_f = pos_alvo[0] - x_futuro
        dy_f = pos_alvo[1] - y_futuro
        ang_alvo_f = math.atan2(dy_f, dx_f)
        erro_angular_futuro = abs(normalizar_angulo(ang_alvo_f - theta_futuro))

        # Escolhe a velocidade que deixa o robô mais bem apontado para o alvo

        if erro_angular_futuro < menor_erro_futuro:

            menor_erro_futuro = erro_angular_futuro

            melhor_v = v_teste

            melhor_w = w_teste

```

```

        w_teste += 0.15

        v_teste += 0.03

# 4. Cinemática Diferencial: Converter velocidade do robô para as rodas
vel_roda_esquerda = (melhor_v - (melhor_w * L / 2.0)) / R
vel_roda_direita = (melhor_v + (melhor_w * L / 2.0)) / R

# Enviar comando de velocidade para a junta
sim.setJointTargetVelocity(motor_esq, vel_roda_esquerda)
sim.setJointTargetVelocity(motor_dir, vel_roda_direita)

# Print de debug no terminal

print(f"Dist: {distancia:.2f}m | Erro Rumo: {math.degrees(erro_angular):.1f}° | V:
{melhor_v:.2f} W: {melhor_w:.2f}", end="\r")

client.step()

except KeyboardInterrupt:

    print("\nSimulação interrompida pelo utilizador.")

finally:

    try:

        sim.setJointTargetVelocity(motor_esq, 0)

        sim.setJointTargetVelocity(motor_dir, 0)

        sim.stopSimulation()

    except:

        pass

    print("Simulação encerrada.")

```

3. Análise do problema que ainda se faz presente

Apesar de o código compilar sem erro e a matemática do arco estar correta no terminal, assim como toda a lógica da janela dinâmica esta implementada, o robô apresentou um comportamento errático na tela do simulador. Não consegui fazer o robô convergir diretamente para o alvo devido a dois grandes problemas práticos:

Inversão do Motor e Batida Crítica

- O terminal mostrava que o Python estava a calcular tudo bem (por exemplo: Dist : 1.65m | Erro Rumo : -94.3°). Ou seja, o programa sabia onde o alvo estava e quanto precisava de virar. Só que, fisicamente, o robô fazia o oposto: andava para trás e girava para o lado errado.
- Analisando a cena, descobri que o motor (MOTOR_ESQUERDO e MOTOR_DIREITO) foi montado na árvore do CoppeliaSim com o eixo local invertido. Como o código enviava um comando positivo esperando que o robô fosse para a frente, o simulador interpretava isso como marcha-atrás. O robô entrou num ciclo de erro: tentava corrigir o rumo, girava ao contrário, afastava-se do Target e ia direto contra o caixote cinzento do mapa, quicando e batendo sem parar por causa do motor gráfico.

O Bug do Robô "Cego" no Início (Lazy Initialization)

- Em vários testes o robô iniciava a simulação completamente travado no sítio, com a velocidade zerada. Ele **só começava a andar se clicasse no Target e o mexesse**. Isto acontece por causa de uma otimização do próprio CoppeliaSim. Para não sobrecarregar o processador do computador, o programa não atualiza ativamente na memória a coordenada global de um objeto do tipo *Dummy* se ele estiver parado e isolado desde o início. O Python lia a posição do alvo como se estivesse estática ou zerada. Quando eu mexia o alvo com o rato, o simulador "acordava", atualizava a matriz e só aí o código recebia o dado real e o robô começava a andar.

Conclusão

Esta sessão prática foi muito importante para perceber que, em robótica, um código matematicamente perfeito falha se não bater certo com a montagem física do robô. A principal conclusão que tirei para o desenvolvimento do projeto foi:

1. Não se confiar cegamente no sinal (+/-) da cinemática diferencial sem antes testar o sentido real de rotação de cada roda no simulador. Preciso de colocar uma chave de inversão no código para corrigir o sentido da junta via software.
2. É obrigatório criar um pequeno loop de aquecimento no início do programa para "forçar" o CoppeliaSim a ler e atualizar a posição do alvo no mapa antes de o robô começar a tomar a decisão de movimento.